

AyED 2025 - CLASE 2



“Primero, resuelve
el problema; luego,
escribe el código”

John Johnson



DevCode.la
#DevQuotes



VISUALGROWTH.COM

Tu participación incrementa tu COEFICIENTE DE PARTICIPACIÓN ACTIVA !!!!

Una de las **competencias** más importantes es la referida al **aprendizaje continuo**.

¿Querés mejorar tus habilidades en técnicas de aprendizaje ?

Te recomendamos algunos MOOCs en Coursera

<https://www.coursera.org/>

1- Aprendiendo a aprender para jóvenes.

<https://www.coursera.org/learn/aprendiendo-a-aprender-para-jovenes>

2- Aprendiendo a aprender.

<https://www.coursera.org/learn/learning-how-to-learn>

No son los únicos, Puedes buscar otros que estén más acordes con tus necesidades.

Otro MOOC muy recomendado:

La mentalidad de crecimiento

<https://www.coursera.org/learn/growth-mindset>



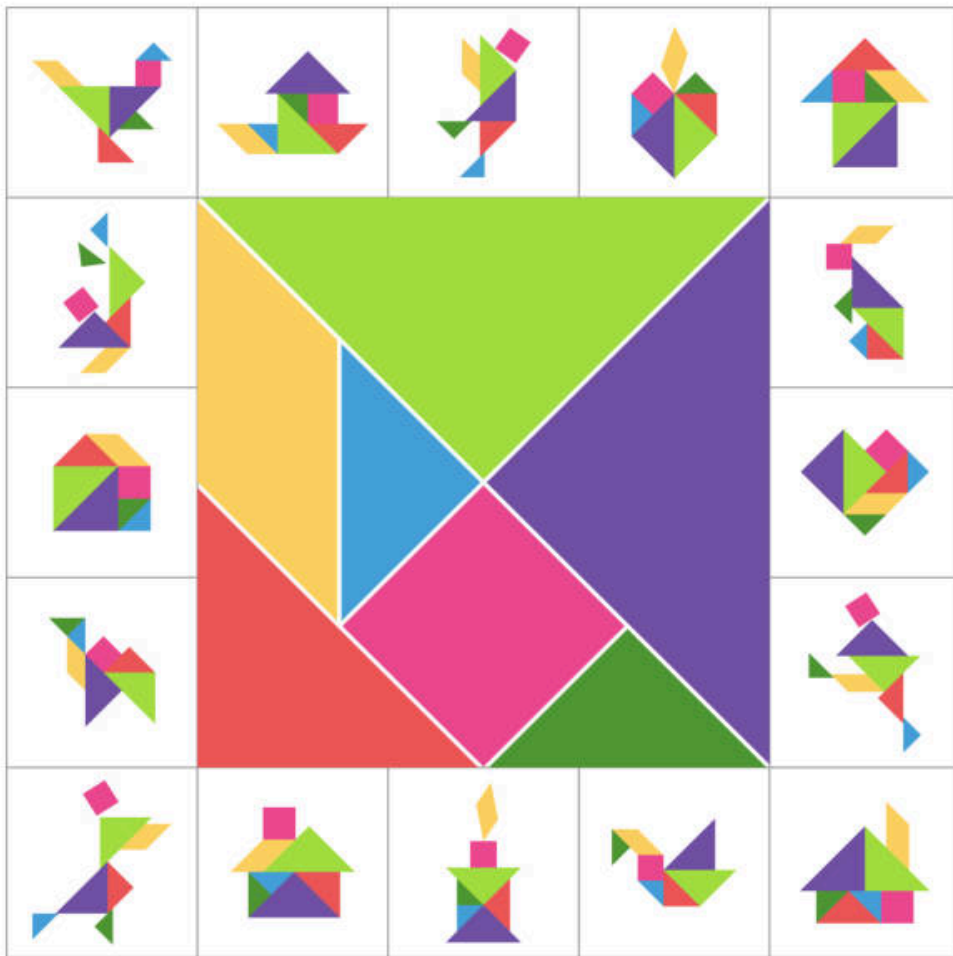
¿Querés mejorar tus habilidades de lógica que te ayuden a resolver problemas ?

Existen muchas actividades y apps para desarrollar esta habilidad. Por ejemplo:

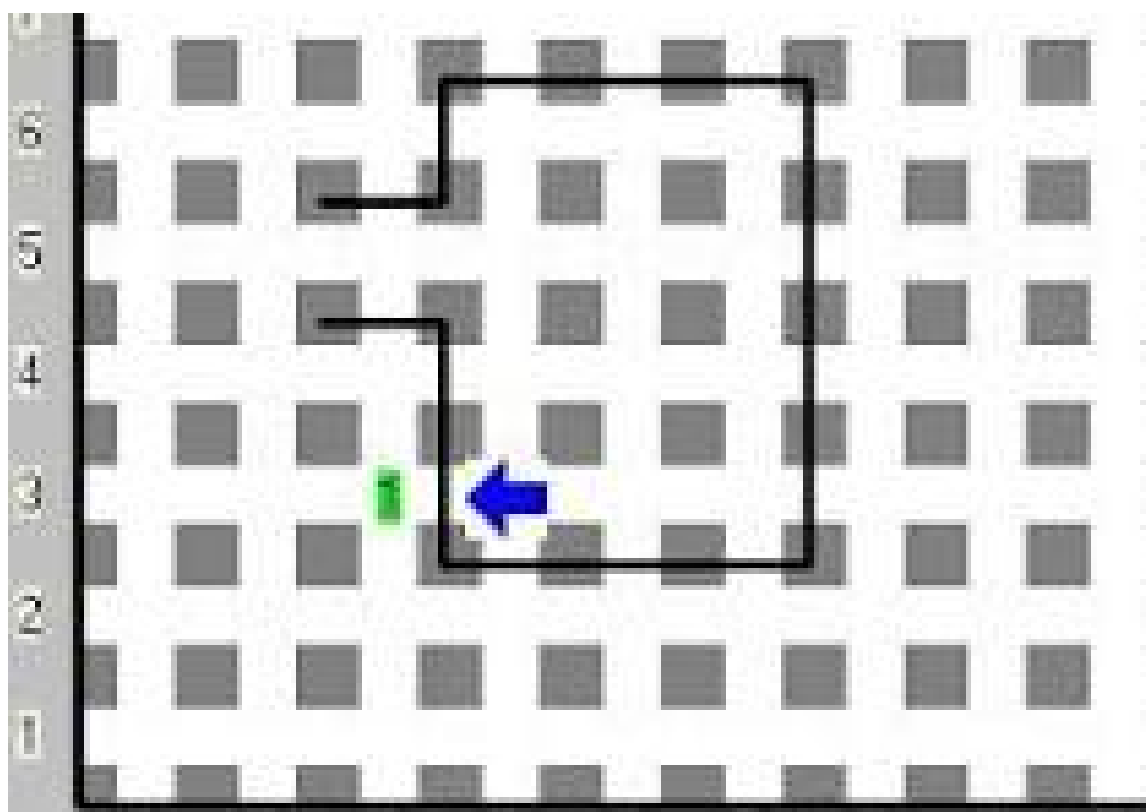
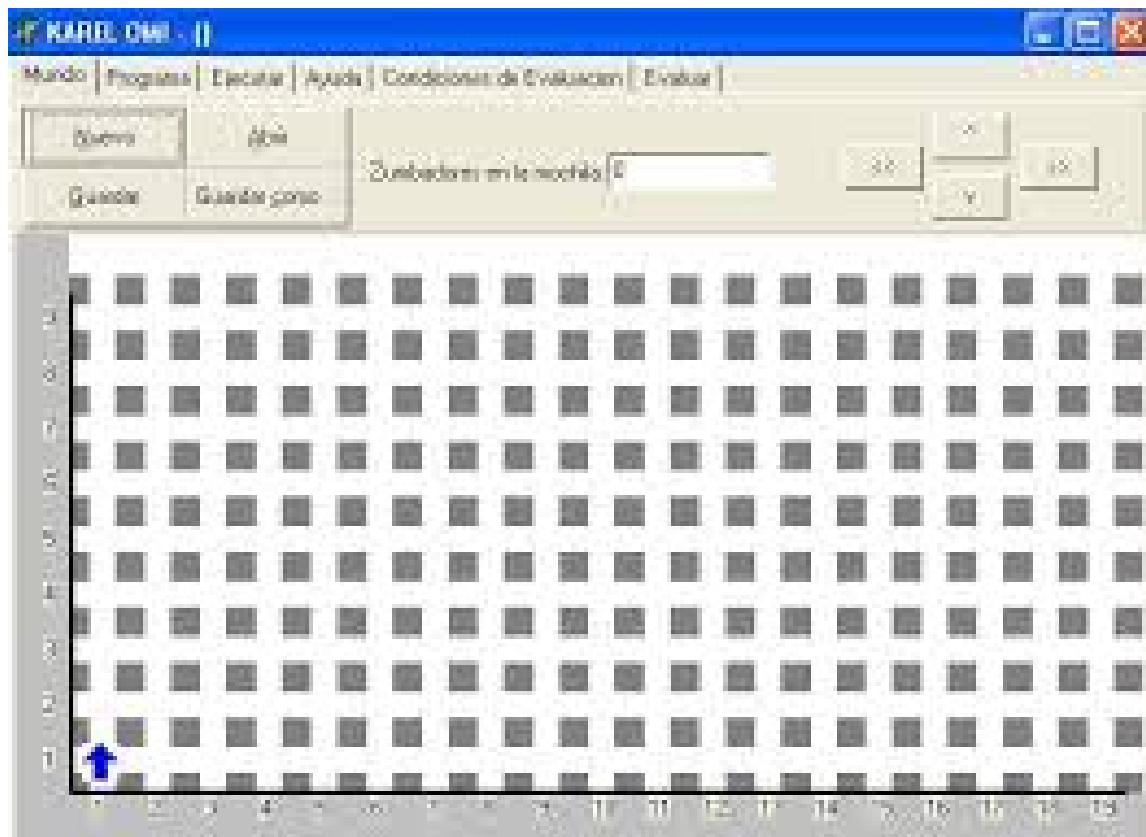
a- Sudoku

5		9				4		
7		8	3		4	9		
6		1				7	3	
4	6	2	5					
3	8	5	7	2		6	4	9
1		7	4		8	2		
2			1					4
		3		4			8	7
	7			5	3			6

b- Tangram



c- Karel el robot



https://www.olimpiadadeinformatica.org.mx/omi/omi/material/karel_el_robot.aspx

Curso de Karel de la universidad de Stanford!!

<https://compedu.stanford.edu/karel-reader/docs/python/en/chapter1.html>

d- Robomind



Curso Robomind:

<https://telmexeducacion.com/course/view.php?id=499>

Y muchos más !!!!

Te sugerimos que trabajes con ellos!!!!
PRACTICA ... PRACTICA ... PRACTICA

EVOCAMOS CONCEPTOS!!!

Pensamiento Computacional

El Pensamiento Computacional es un **modelo de resolución de problemas que utiliza herramientas propias del ámbito de la informática**, aunque puede aplicarse a diversos contextos o ámbitos.

Es un concepto que se entiende como una manera de pensar diferente **que no se restringe al código y la programación**, sino como una serie de habilidades analíticas de razonamiento lógico y sistemático para llegar a una solución dada.

En este proceso, se ponen en juego una serie de estrategias cognitivas aplicadas a la resolución de problemas:

La abstracción: centrar la atención solo en la información importante, ignorando los detalles irrelevantes.

La descomposición: dividir un problema o sistema complejo en partes más pequeñas y manejables.

El reconocimiento de patrones: identificar objetos o comportamientos que son recurrentes y clasificarlos.

El pensamiento algorítmico: desarrollar una solución paso a paso para resolver un problema.

Estas habilidades se complementan y permiten abordar problemas complejos y resolverlos teniendo en cuenta las partes que los componen y las resoluciones a problemas similares realizados anteriormente.

COMPONENTES DEL PENSAMIENTO COMPUTACIONAL

Descomposición



Desarmar el problemas en partes, realizando una descomposición en subproblemas facilitando su resolución.

Tomar decisiones en cuanto al manejo de las subtareas teniendo en cuenta la integración.

Generalización



Identificar patrones, regularidades y conexiones.

Resolver nuevos problemas basados en problemas ya resueltos; utilizando la solución general.

Abstracción



Detectar elementos claves en un problema ignorando los detalles irrelevantes. Elegir una forma de representación.

Pensamiento Algorítmico



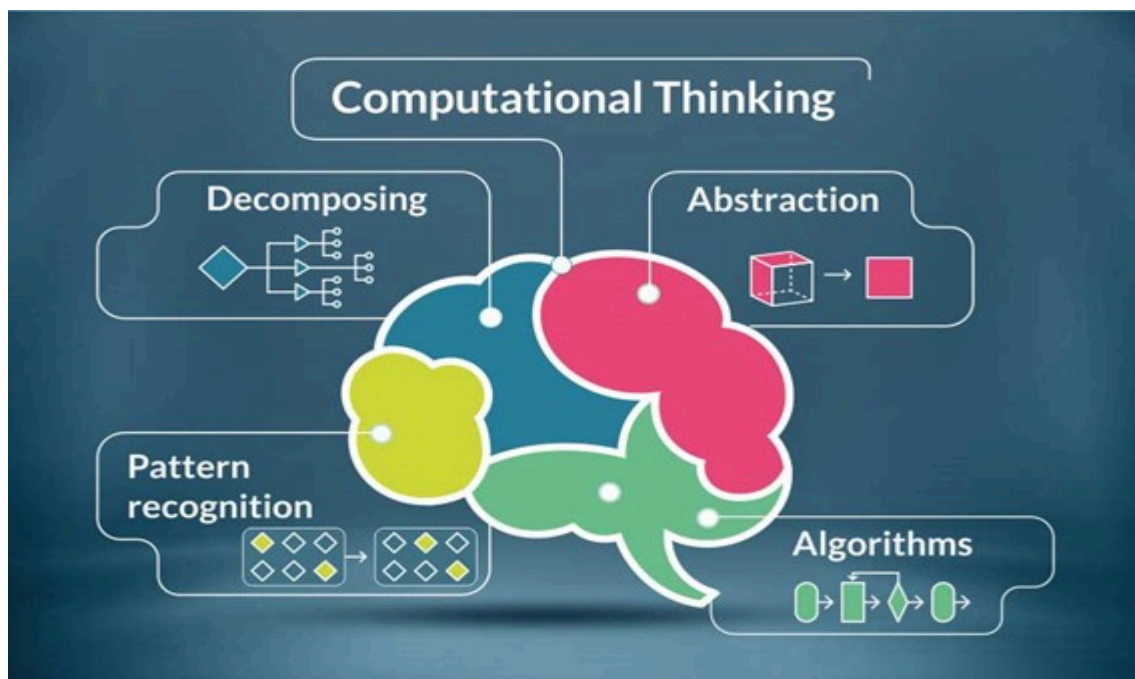
Pensar el problema como una secuencia de pasos ordenados. Crear y/o ejecutar algoritmos.

Evaluación



Encontrar la mejor solución; tomando decisiones sobre el buen uso de los recursos, para alcanzar un propósito.

Analizar y re-evaluar durante todas las etapas del proceso.



PASOS PARA RESOLVER UN PROBLEMA UTILIZANDO EL PENSAMIENTO COMPUTACIONAL

1- **Comprender el problema** es una etapa del pensamiento computacional que consiste en identificar qué se pide, sin considerar las limitaciones del problema.

El pensamiento computacional es un conjunto de habilidades que permiten resolver problemas de manera lógica y estructurada. Se basa en la descomposición de problemas complejos en partes más pequeñas, para poder abordarlos de manera más eficiente.

2- Una vez completado el punto anterior de una forma clara, podemos utilizar algunas de estas recomendaciones:



Veamos esto en formato pseudocódigo: (¿recuerdan al algoritmo del cambio de la lámpara ?)

comprendimos = “ ”

Repetir hasta comprendimos = cierto

 Leer-Comprender el problema

 Si comprendimos-problema

 comprendimos = cierto

 Finsi

Fin Repetir

Resolver-Problema

¿Qué nos diría la IA sobre cómo resolver problemas con la ?

¡Gran pregunta! El **pensamiento computacional** es una forma de resolver problemas de manera estructurada y eficiente, **inspirada en cómo piensan los programadores** , pero aplicable a la vida diaria, la ciencia y cualquier área. 🚀

Aquí te explico **cómo usarlo paso a paso** para resolver cualquier problema:

✅ 1. Descomposición (Dividir el problema)

👉 ¿Qué es?

Separar un problema grande en partes más pequeñas y manejables.

Ejemplo:

Problema: Quiero organizar un torneo de fútbol escolar.

Descomposición:

- Hacer la lista de equipos.
- Definir el fixture.
- Alquilar la cancha.
- Comprar premios.
- Difundir el evento.

✓ **Ventaja:** Hace que un problema grande deje de parecer imposible.

✓ **2. Abstracción (Filtrar lo esencial)**

¿Qué es?

Centrarse en la información más importante y dejar de lado los detalles irrelevantes.

Ejemplo:

Para programar un semáforo, no importa el color exacto de los autos, solo importa su presencia o ausencia.

✓ **Ventaja:** Evita distraerte con detalles que no ayudan a resolver el problema.

✓ **3. Reconocimiento de patrones**

¿Qué es?

Buscar similitudes o repeticiones que ayuden a predecir soluciones o a usar una fórmula general.

Ejemplo:

- Notar que todos los días el tráfico aumenta a la misma hora.
- Observar que cada juego de un torneo dura 30 minutos.

✓ **Ventaja:** Permite usar soluciones ya conocidas o reutilizables.

✓ 4. Diseño de algoritmos (Pasos ordenados para la solución)

👉 ¿Qué es?

Crear una **secuencia clara de pasos** que resuelva el problema de forma efectiva.

🔍 Ejemplo:

Algoritmo para hacer un té:

1. Calentar agua.
2. Poner la bolsita de té en la taza.
3. Verter el agua caliente.
4. Esperar 3 minutos.
5. Sacar la bolsita.
6. Endulzar si se desea.

✓ **Ventaja:** Cualquier persona (o computadora) puede seguir los pasos y obtener el mismo resultado.

✓ 5. (Opcional) Generalización y automatización

👉 ¿Qué es?

Crear una solución que se pueda **repetir o automatizar** para problemas similares.

🔍 Ejemplo:

Una app de reparto no solo resuelve un pedido, sino **automatiza** el proceso de asignar repartidores a cualquier cantidad de pedidos.

📌 **Ejemplo de Aplicación Completa: Resolver “¿Cómo llegar a la universidad?”**

1. **Descomposición:** ¿Qué transporte usar? ¿Cuál es la ruta?
¿Cuánto tiempo toma cada tramo?
 2. **Abstracción:** No importa el color del bus, solo la línea que me deja más cerca.
 3. **Patrones:** Notar que el tráfico siempre es peor a las 8:00 am.
 4. **Algoritmo:**
 - Salir a las 7:30 am.
 - Tomar el bus 42.
 - Bajar en la parada de la universidad.
 5. **Generalización:** Si otro día tengo que ir a otro lugar, puedo repetir el análisis.
-

Resumen Final - ¿Cómo usar el Pensamiento Computacional?

- ✓ Divide el problema en partes pequeñas.
- ✓ Quédate solo con lo esencial.
- ✓ Busca patrones.
- ✓ Diseña un paso a paso (algoritmo).
- ✓ Piensa si se puede aplicar o automatizar para otros problemas similares.

 ¿Quieres un ejemplo aplicado a un área específica? (educación, negocios, programación, etc.) ¡Te lo preparo! 

Ejemplo: cómo resolver una ecuación de segundo grado aplicando PC?

¡Excelente! Resolver una **ecuación de segundo grado** es un buen ejemplo de cómo aplicar **pensamiento computacional** usando **algoritmos y descomposición**.

La forma general de la ecuación es:

$$ax^2+bx+c=0 \quad ax^2 + bx + c = 0$$

✓ Aplicando Pensamiento Computacional Paso a Paso

1 Descomposición del Problema

👉 ¿Qué necesito hacer?

- Calcular el **discriminante**: $\Delta = b^2 - 4ac$
- Evaluar si hay solución real o no.
- Aplicar la **fórmula cuadrática**:

$$x = \frac{-b \pm \sqrt{\Delta}}{2a}$$

2 Abstracción (qué datos son esenciales)

- Solo necesito los valores de **a**, **b** y **c**.
 - No importa el nombre de la ecuación o de dónde viene.
-

3 Reconocimiento de patrón

- Todas las ecuaciones de segundo grado se resuelven con la misma **fórmula general**.
-

4 Diseño del algoritmo (paso a paso)

Entrada: valores de a, b y c.

Proceso:

1. Calcular $\Delta = b^2 - 4ac$
2. Si $\Delta < 0$: No hay solución real.
3. Si $\Delta = 0$: Una solución $x = \frac{-b}{2a}$
4. Si $\Delta > 0$: Dos soluciones reales:

$$x_1 = \frac{-b + \sqrt{\Delta}}{2a} \quad x_2 = \frac{-b - \sqrt{\Delta}}{2a}$$

Salida: las soluciones de x.

✓ Ejemplo concreto con valores:

Resolver $2x^2 - 4x - 6 = 0$

1. **a = 2, b = -4, c = -6**
2. Calculo el discriminante:

$$\Delta = (-4)^2 - 4 \cdot 2 \cdot (-6) = 16 + 48 = 64 \quad \Delta = (-4)^2 - 4 \cdot 2 \cdot (-6) = 16 + 48 = 64$$

3. Como $\Delta > 0$, hay dos soluciones:

$$x_1 = \frac{-(-4) + \sqrt{64}}{2 \cdot 2} = \frac{4 + 8}{4} = 3 \quad x_2 = \frac{-(-4) - \sqrt{64}}{2 \cdot 2} = \frac{4 - 8}{4} = -1$$

✓ **Resultado:** Las soluciones son $x_1 = 3$ y $x_2 = -1$

✓ 5 Generalización / Automatización

Este algoritmo sirve para **cualquier ecuación cuadrática** que te den.

✓ Puedes programarlo en una calculadora, app o en Python, por ejemplo.

Conclusión:

La resolución de una ecuación de segundo grado es un ejemplo perfecto de **pensamiento computacional**, porque sigues un **algoritmo claro**, repites una **estructura generalizada** y llegas a la solución con **lógica paso a paso**.

 ¿Quieres que te dé el código en Python o en pseudocódigo de este algoritmo?

Ejemplo ejercicio de Bebras

¡Perfecto! Aquí tienes un **ejercicio estilo Bebras** para alumnos de **18 años**, con solución detallada y destacando la **habilidad de pensamiento computacional** que se trabaja.

Ejercicio Bebras - Nivel 18 años

Título: La Fábrica de Códigos

Habilidad: Abstracción - Algoritmos - Reconocimiento de patrones

Problema:

Una fábrica genera códigos de acceso siguiendo este patrón:

- Toma un número de 4 dígitos.
- Cambia cada dígito por la suma de ese dígito y su posición (la primera posición es 1).
- Si el resultado es mayor o igual a 10, se queda solo con el dígito de las **unidades**.

Ejemplo: Si el número es **3157**:

- **3** (posición 1) $\rightarrow 3 + 1 = 4$
- **1** (posición 2) $\rightarrow 1 + 2 = 3$
- **5** (posición 3) $\rightarrow 5 + 3 = 8$
- **7** (posición 4) $\rightarrow 7 + 4 = 11 \rightarrow 1$ (se toma la unidad)

Código final: **4381**

Pregunta:

¿Cuál será el código generado por la fábrica si el número de entrada es **4728**?

Resolución paso a paso (Algoritmo aplicado):

- Primer dígito **4** (posición 1): $4 + 1 = 5$
- Segundo dígito **7** (posición 2): $7 + 2 = 9$
- Tercer dígito **2** (posición 3): $2 + 3 = 5$
- Cuarto dígito **8** (posición 4): $8 + 4 = 12 \rightarrow$ solo la unidad $\rightarrow 2$

 **Código final: 5952**

Respuesta Correcta: 5952

Habilidad Computacional que se trabaja:

✓ **Abstracción:** Ignorar el número original y quedarse solo con el algoritmo de posición y suma.

✓ **Reconocimiento de patrones:** Notar la repetición de la suma posición + valor.

✓ **Algoritmos:** Aplicar una secuencia clara de pasos para transformar el dato.

COMPARTIMOS SOLUCIONES DE LA ACTIVIDAD ANTERIOR

- **VASOS**
- **LIGHTBOT**
- **BEBRAS (subidos a Socrative)**
- **ALGORITMOS DE LOS BAILES**

ACTIVIDAD

CON PEER - LEARNING (APRENDIZAJE ENTRE PARES)

0 En el documento ACTIVIDAD PEER LEARNING 1, encontrarán los enunciados a los problemas.

1 Ingresar a Socrative.

2 Aula: UTNVM

3 En nombre ingresar Apellido y Nombre , en este orden.

4 Resolver el problema 1 en forma INDIVIDUAL. (Tiempo: 3min)

5 Resolver el problema 1 en forma GRUPAL (tiempo : 2 min)

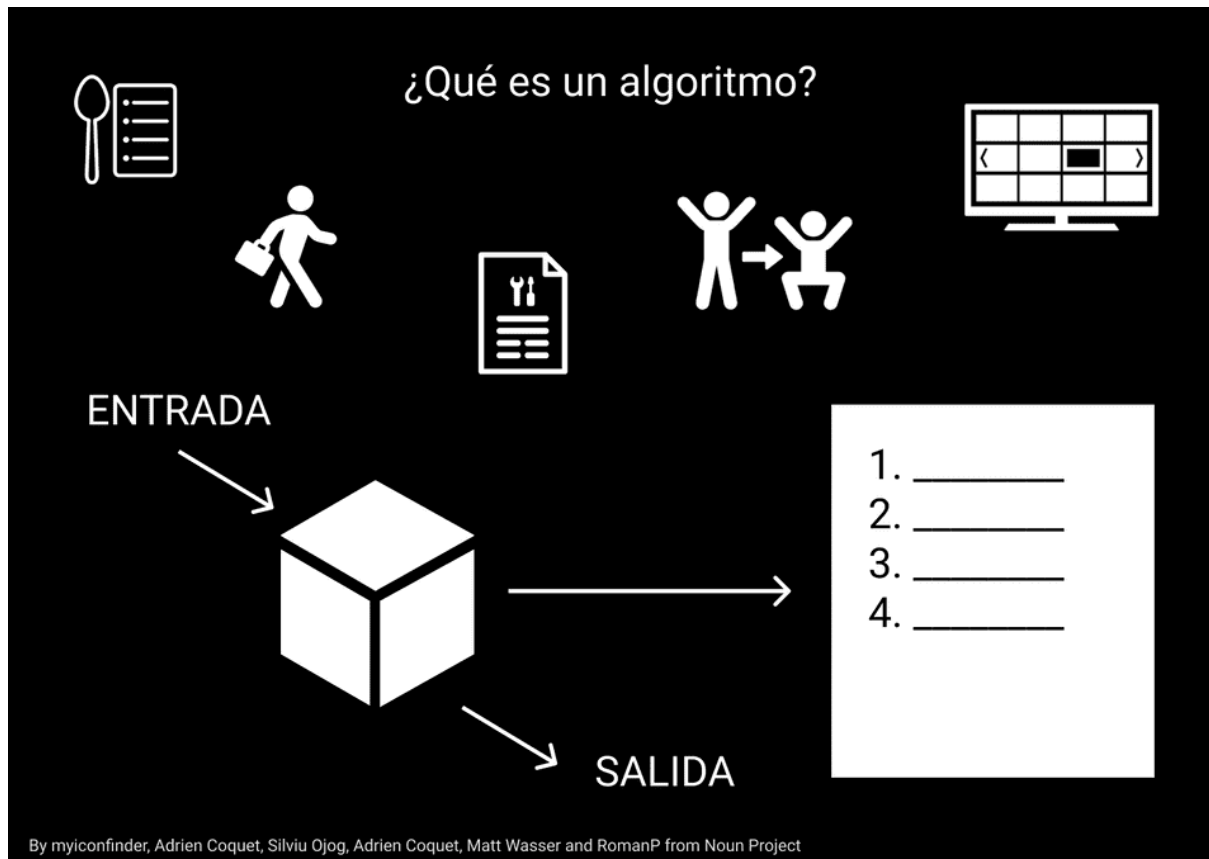
6 Resolver el problema 2 en forma INDIVIDUAL (tiempo 3 min)

7 Resolver el problema 2 en forma GRUPAL. (tiempo 2 min)

8 Resolver el problema 3 en forma INDIVIDUAL . (tiempo 3 min)

9 Resolver el problema 3 en forma GRUPAL . (tiempo 2 min)

Afinando el concepto de Algoritmo !!.

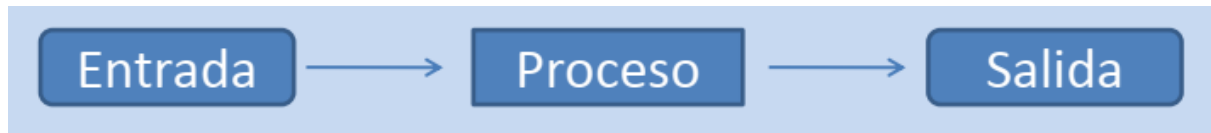


ALGORITMO:

- Un **algoritmo** se puede definir como una secuencia de pasos o instrucciones que representan un modelo de solución para determinado tipo de problema (Joyanes Aguilar).
- Características de un algoritmo:
 - + **Preciso**: indicar orden de operaciones
 - + **Definido**: si se sigue un algoritmo 2 veces, se debe obtener el mismo resultado.
 - + **Finito**: Debe finalizar en algún momento.

ESTRUCTURA DE UN ALGORITMO

Todo algoritmo consta de 3 secciones principales:



Entrada: es la introducción de datos para ser operados.

Proceso: Es el conjunto de operaciones a realizar para la solución del problema.

Salida: Son los resultados obtenidos a través del proceso.

Las estructuras básicas de un algoritmo son:

1- Secuenciales: implica escribir un paso tras otro. Se ejecutan en el orden de aparición.

Por ejemplo:

Leer n1

Leer n2

suma = n1 + n2

imprimir suma

2- Selectivas

Se utilizan para tomar decisiones. Evalúa una condición, si la condición es verdadera, realiza la acción indicada, en caso contrario no la ejecuta.

si condición entonces
 acción-es
finsi

Existen diferentes tipos de selectivas: simple, doble, múltiple.

3- Repetitivas (iterativas)

Este tipo de estructura se utiliza para ejecutar acciones repetidamente dependiendo de una condición o una cantidad fija de veces.

Existen diferentes tipos de estructuras repetitivas: Mientras, Para, Repetir, etc.

Hacer mientras condición
 Acción 1
 Acción 2
 ...
 Acción n
Fin-mientras

Repetir n veces
 Acción 1
 Acción 2
 ...

Acción n
Fin-repetir

Teorema de Böhm-Jacopini

El teorema de Böhm-Jacopini es un principio formulado en el año 1966 por los científicos italianos Corrado Böhm y Giuseppe Jacopini. Este **establece que cualquier algoritmo puede ser implementado usando solo tres estructuras básicas de control: la secuencia, la selección y la iteración**. Es decir, sin importar la complejidad de un programa, siempre se puede construir usando estas tres estructuras.

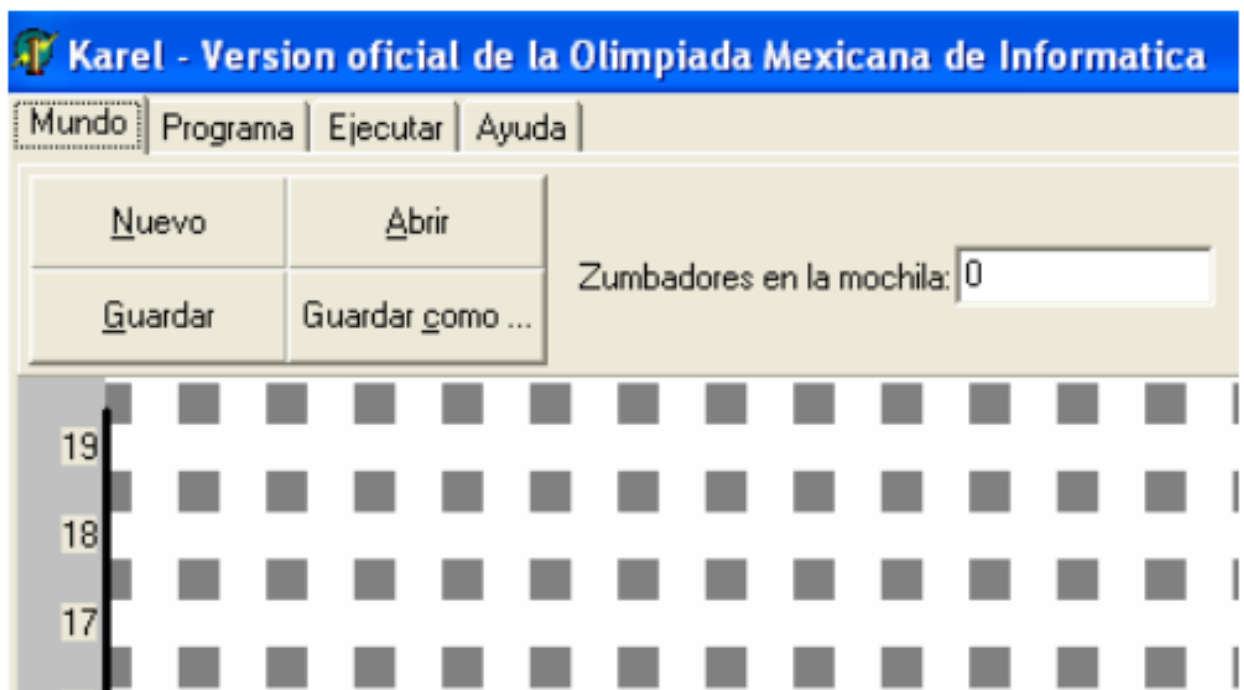
Manos a la obra!!

Nuestros primeros algoritmos usando el robot Karel.

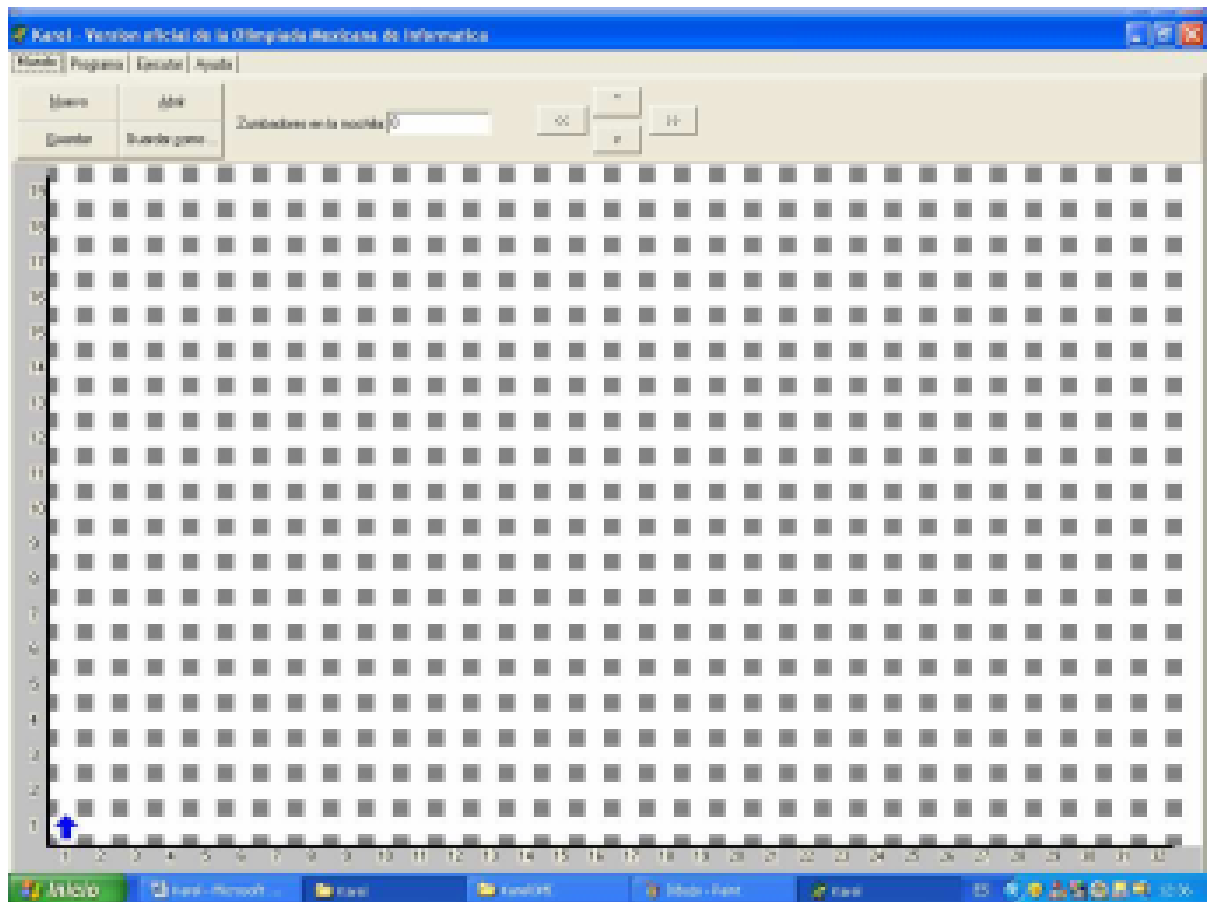
Descargalo desde:

https://www.olimpiadadeinformatica.org.mx/omi/omi/material/karel_el_robot.aspx

Pantalla principal de Karel:



El mundo de Karel:



Karel es la simulación de un robot el cual tiene las siguientes limitantes:

- No puede dar vueltas hacia la derecha
- No tiene reversa
- No salta paredes
- Solo detecta a los zumbadores que están en su misma coordenada
- No sabe cuántos zumbadores hay en su mochila
- Una vez que se apaga no se puede volver a prender salvo que vuelva a comenzar el programa
- No tiene variables explícitas a diferencia de otros lenguajes

Sintaxis básica:

Karel cuenta con cinco instrucciones básicas para hacer todas sus tareas. Las instrucciones son las siguientes:

avanza: Karel avanza una cuadra hacia donde esté orientado. Si hay una pared enfrente, el programa marcará un error y dejará de ejecutarse.

gira-izquierda: Karel gira hacia la izquierda 90°, cambiando su orientación.

coge-zumbador: Karel recoge un zumbador en el lugar donde está parado. Si no hay zumbador en ese lugar, el programa marcará error y termina la ejecución.

deja-zumbador: Karel deja un zumbador en el lugar que está parado.

Si Karel no tiene zumbadores en la mochila, entonces no podrá dejar el zumbador y el programa marcará error.

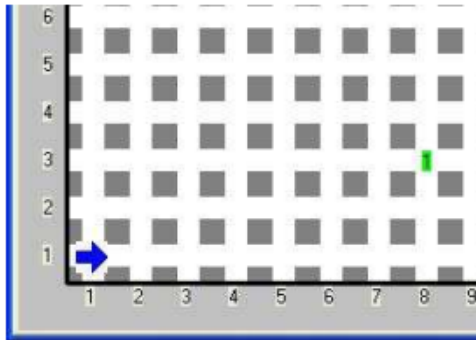
apagate: Finaliza la ejecución del programa. Karel ya no podrá hacer más cosas porque ya está apagado.

Ejemplos :

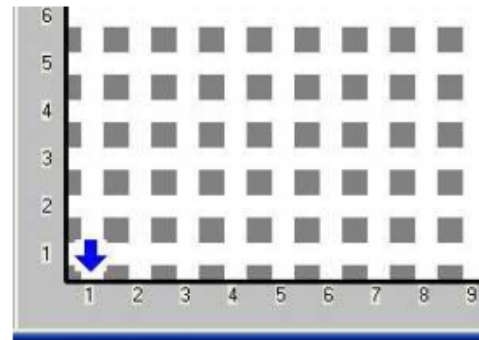
```
1 iniciar-programa
2     inicia-ejecucion
3         apagate;
4     termina-ejecucion
5 finalizar-programa
6
```

Problema 1.

Karel se encuentra en su casa (posición 1,1 viendo hacia el este), se ha dado cuenta que es hora de ir a recoger a su hermana a su escuela (posición 8,3), su misión es ir por ella y regresar juntos a casa.



Estado inicial, Karel tiene 0 zumbadores



Estado final, Karel tiene 1 zumbador y en la posición inicial

Programa

iniciar-programa

inicia-ejecucion

avanza;

avanza;

avanza;

avanza;

avanza;

avanza;

avanza;

gira-izquierda;

avanza;

avanza;

coge-zumbador;

gira-izquierda;

avanza;

avanza;

avanza;

avanza;

avanza;

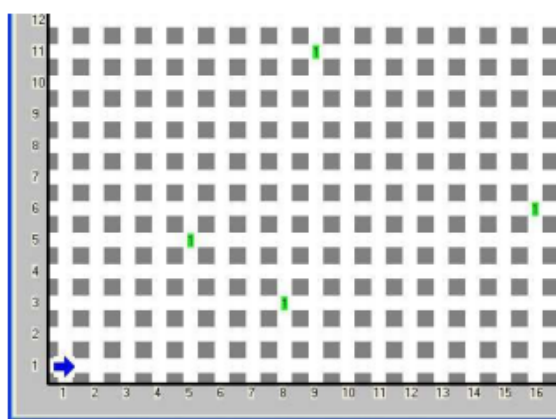
avanza;
avanza;
gira-izquierda;
avanza;
avanza;
apagate;
termina-ejecucion
finalizar-programa

Problema 2.

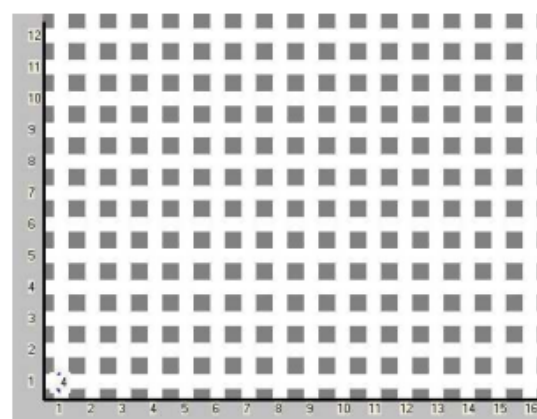
En un día de compras, Karel sale de su casa (posición 1,1) y se dirige a obtener los siguientes productos: leche, pan, huevo y carne, cada uno en diferentes establecimientos, al final regresa a casa con todas sus compras

- En el mundo de Karel los establecimientos donde venden esos productos están en las siguientes posiciones:

- o Leche: posición (5, 5).
- o Pan: posición (9, 11).
- o Huevo: posición (16, 6).
- o Carne: posición (8, 3).



Estado inicial, Karel tiene 0 zumbadores



Estado final, Karel tiene 4 zumbadores y en la posición inicial

iniciar-programa
inicia-ejecucion
avanza;
avanza;
avanza;
avanza;
gira-izquierda;
avanza;
avanza;
avanza;
avanza;
coge-zumbador;
gira-izquierda;
gira-izquierda;
gira-izquierda;
avanza;
avanza;
avanza;
gira-izquierda;
gira-izquierda;
gira-izquierda;
avanza;
avanza;
coge-zumbador;
gira-izquierda;
avanza;
gira-izquierda;
avanza;
avanza;
avanza;
avanza;
avanza;
avanza;
avanza;

avanza;
coge-zumbador;
gira-izquierda;
gira-izquierda;
gira-izquierda;
avanza;
avanza;
avanza;
avanza;
avanza;
avanza;
avanza;
gira-izquierda;
gira-izquierda;
gira-izquierda;
avanza;
avanza;
avanza;
avanza;
avanza;
coge-zumbador;
avanza;
avanza;
avanza;
avanza;
avanza;
gira-izquierda;
gira-izquierda;
gira-izquierda;
avanza;
avanza
avanza;
avanza;
avanza;

avanza;
avanza;
avanza;
avanza;
avanza;
avanza;
avanza;
avanza;
avanza;
avanza;
deja-zumbador;
deja-zumbador;
deja-zumbador;
deja-zumbador;
apagate;
termina-ejecucion
finalizar-programa

Problema a resolver :

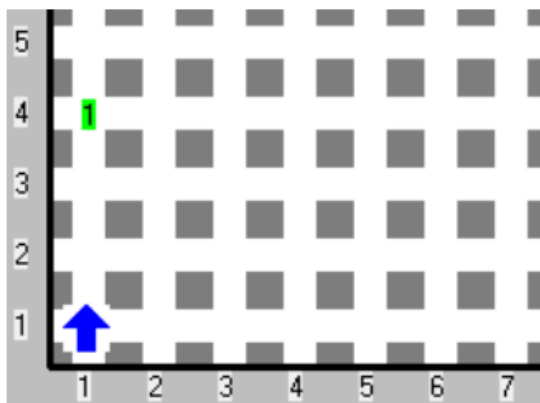
Ayuda a Karel a ir por su cartera a casa de Juan.

Consideraciones:

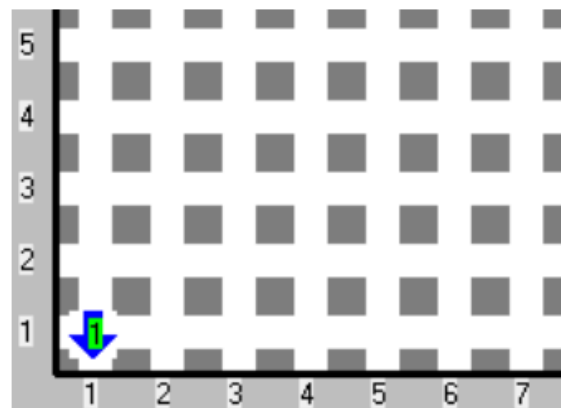
- Karel siempre empieza en su casa viendo hacia el norte.
- La casa de Juan esta a tres casas de la suya hacia el norte (posición 4,1).
- No existen obstáculos de por medio.
- La cartera es representada por un zumbador.
- Karel debe finalizar en su casa sin importar su orientación.
- Karel deberá dejar la cartera en su casa. (fila 1 , columna 1)

Ejemplo:

Entrada



Salida



Pregunta?

Que tipo de estructura usamos para resolver los ejercicios ?

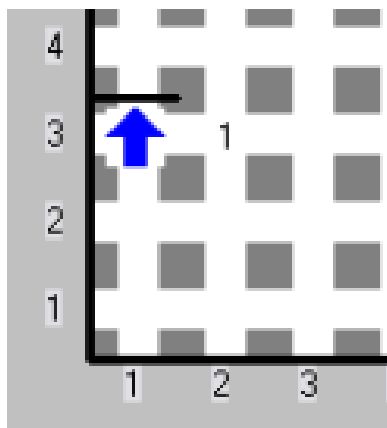
Incorporamos estructura de decisiones simples en Karel

Hay veces que Karel puede hacer una serie de instrucciones si se cumple una condición previamente establecida. Por ejemplo:

```
si frente-libre entonces inicio
    avanza;
fin;
```

En las líneas anteriores se muestra una pequeña decisión que va a hacer Karel. Si en frente de él no hay pared, entonces avanza; si hay pared entonces no hace nada.

Problema: Karel está en su casa orientado hacia el norte y deberá caminar en ese sentido, sus presentimientos le dicen existe una pared a 2 calles al norte pero el deberá de constatarlo, al llegar a esa posición y si en verdad existe la pared deberá girar a la derecha, avanzar una calle y apagarse.



Código:

iniciar-programa

inicia-ejecucion

avanza;

avanza;

si frente-bloqueado entonces inicio

 gira-izquierda;

 gira-izquierda;

 gira-izquierda;

 avanza;

fin;

apagate;

termina-ejecucion

finalizar-programa

Tabla de condiciones en Karel:

<i>frente-libre</i>	<i>junto-a-zumbador</i>	<i>orientado-al-este</i>
<i>frente-bloqueado</i>	<i>no-junto-a-zumbador</i>	<i>orientado-al-oeste</i>
<i>izquierda-libre</i>	<i>algun-zumbador-en-la-mochila</i>	<i>no-orientado-al-norte</i>
<i>izquierda-bloqueada</i>	<i>ningun-zumbador-en-la-mochila</i>	<i>no-orientado-al-sur</i>
<i>derecha-libre</i>	<i>orientado-al-norte</i>	<i>no-orientado-al-este</i>
<i>derecha-bloqueada</i>	<i>orientado-al-sur</i>	<i>no-orientado-al-oeste</i>

Incorporamos estructura de repeticiones en Karel

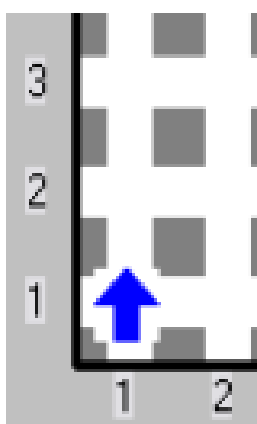
Karel dispone de diversas estructuras de repetición:

Repeticiones fijas: Ejecutarán el código UN determinado número de veces (el número de veces no puede variar mientras se ejecuta el programa).

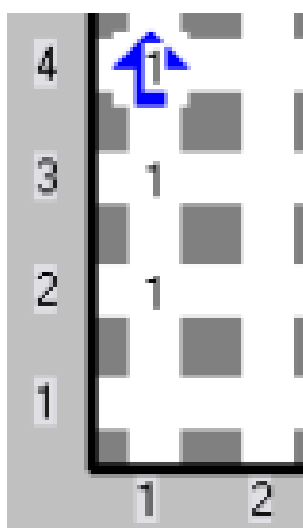
Repeticiones condicionales: Ejecutan el código indeterminado número de veces (depende de que tantas veces se cumpla la condición)

Ciclo anidado: Ocurre cuando un bloque de instrucciones que se repetirá se encuentra dentro de otro ciclo con más instrucciones por repetir.

```
iniciar-programa
inicia-ejecucion
  repetir 3 veces
    inicio
      avanza;
      deja-zumbador;
    fin;
  apagate;
termina-ejecucion
finalizar-programa
```

Situación inicial



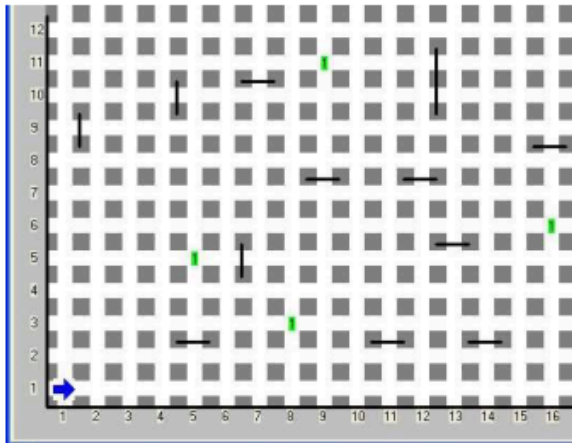
Situación final

ACTIVIDAD PREVIA A LA PRÓXIMA CLASE

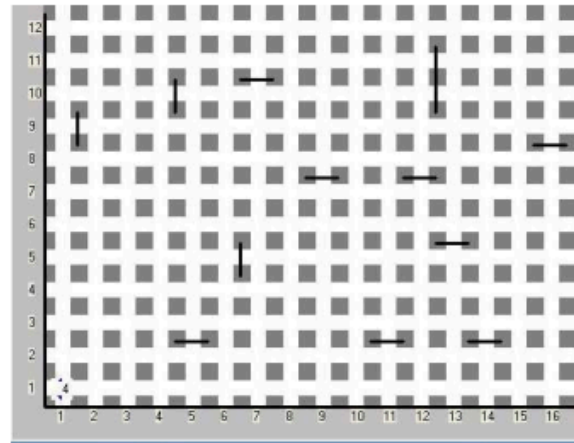
1- Resolver los siguientes problemas con Karel.

Problema a. Es el día de compras, Karel tiene que ir por los productos de siempre, sin embargo algunas calles están cerradas, por lo que tiene que tomar vías alternas.

- Los establecimientos están en las mismas posiciones.

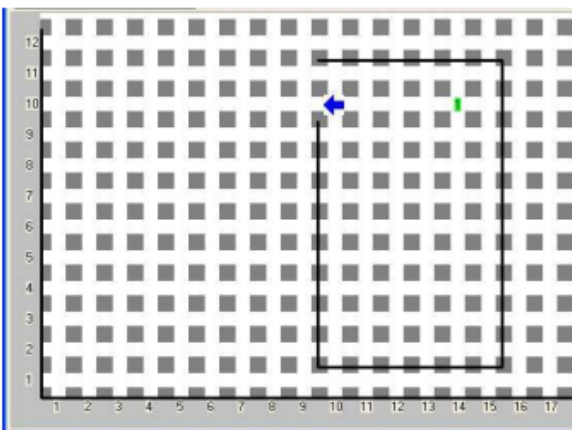


Estado inicial, Karel tiene 0 zumbadores

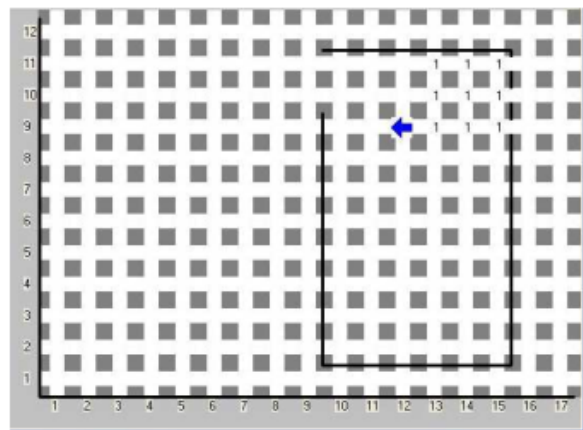


Estado final, Karel tiene 4 zumbadores y en la posición inicial

Problema b. Karel tiene una mascota que se le ha escapado en varias ocasiones, preocupado porque la última vez le costó mucho trabajo encontrarlo, decidió encerrarlo (utilizando 8 zumbadores) para que no se volviera a escapar.



Estado inicial



Estado final

2- Revisar del libro Fundamentos de Programación (Joyanes), el capítulo 2.